Introduction to Large Language Models

From next-token prediction to reasoning and agentic AI

Luigi Sigillo · Postdoctoral Fellow · Petroni Lab · EMBL Rome

EMBL

A tour of the machine behind the text box

1. What exactly is being predicted?
2. Why did Transformers change the field?
3. How does a raw model become an assistant?
4. What breaks — and how do we fix it?
5. How can scientists use this safely and effectively?

What is a language model?

The task

- Given all previous tokens, predict the next one.

the	mouse	ate	the	?

- The model outputs a probability distribution over the whole vocabulary.

- **Training objective:** minimise cross-entropy (*one number, lower is better.)* on shifted text.

The surprising part

- To autocomplete well across the whole internet the model must learn grammar, facts, reasoning — and world structure.

- Scale this simple objective to trillions of tokens and capability emerges.



https://tiktokenizer.vercel.app/

What is a token?

The model never sees words or letters — it sees tokens: sub-word chunks turned into numbers.

“TP53 regulates the cell cycle.” → split into tokens:



Small grey numbers are the integer token IDs. Common words are one token; “TP53” splits into two.

Why this matters?

- “Predict the next token” is the whole game, **a token**, not a word or a letter.
- Because it sees chunks, not characters, it can struggle to count or spell letters.

We’ll see the spelling failure on the next slide.

When tokenization bites

The model reasons over tokens, not characters — so simple-looking tasks can fail.

“How many r’s in strawberry?”

st | raw | berry

The letters are hidden inside chunks — it can’t easily “see” each r.

✗ Model often says “2” ✓ Answer: 3

“Which is bigger, 9.11 or 9.9?”

“11” and “9” arrive as separate tokens; the model can treat them like version numbers or page numbers.

✗ Often says 9.11 ✓ 9.9 is bigger

(A known, much-cited failure mode.)

The lesson for scientific use:

These aren’t “stupid” mistakes — they are a direct consequence of how text is tokenized.

- Don’t trust the model for character counting, exact arithmetic, or precise string edits.
- For anything exact — sequence positions, coordinates, maths — give it a tool or code

Why does next-token prediction produce intelligence?

A model that predicts the next token must implicitly learn:

- **Grammar** — what sequences are valid.
- **Facts** — what entities exist and co-occur.
- **World structure** — how objects, events and people behave.
- **Reasoning patterns** — what conclusions usually follow from evidence.

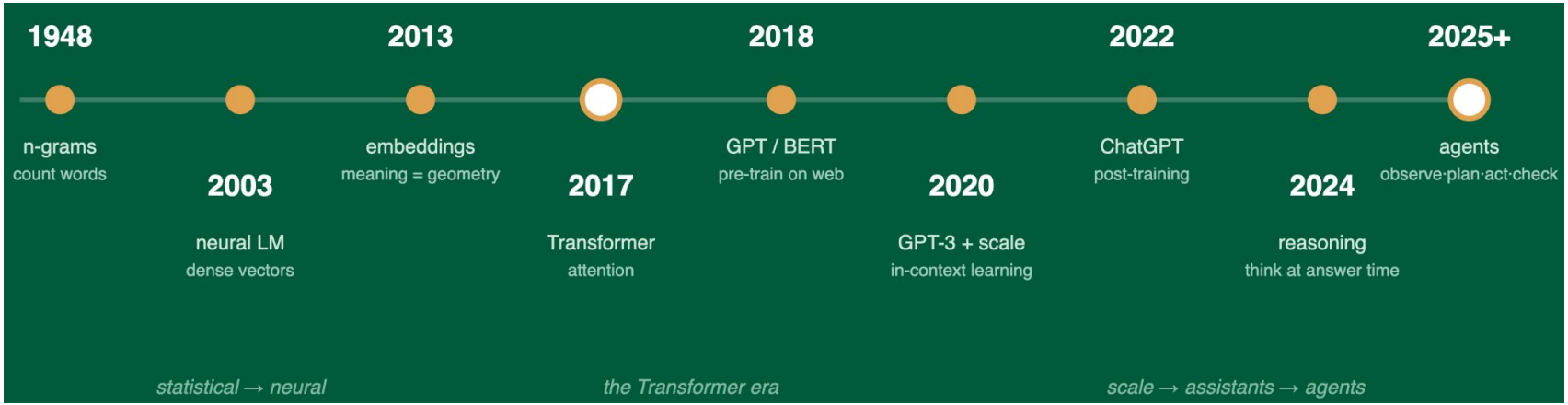
Example: *"The glass fell from the table and ..."*

Possible continuations depend on understanding: gravity, objects, causality, physical consequences

- The model is never taught physics directly. It learns whatever structure helps reduce prediction error.
- Prediction is the objective. World modelling emerges as a consequence.

One field, eight turning points

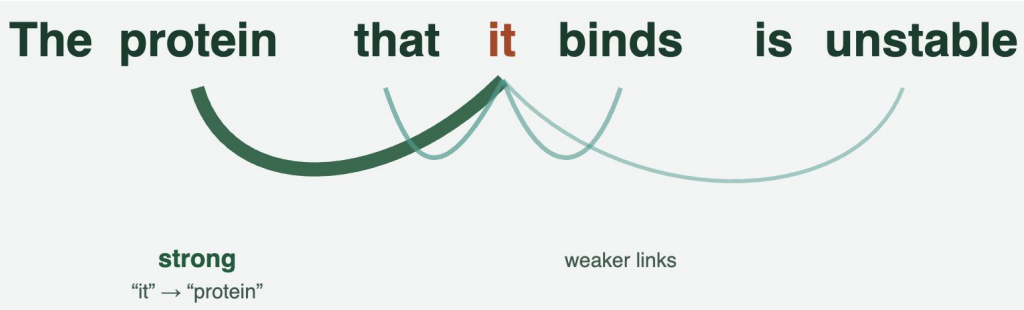
One idea — predict the next token — taken very seriously.



Attention, before the math

Every word looks at every other word and decides which ones matter.

To predict what follows, the word “it” must find what it refers to:



The model learns these weights — nobody hand-codes that “it” means “protein.”

Biology analogy: a Multiple-Sequence Alignment

- Each position scans the whole sequence and weights the other positions by how relevant they are, just like attention.

The difference: in an MSA the scoring is fixed; in a Transformer the model learns what to align to.

2017 · THE HINGE

Attention Is All You Need

Why it mattered

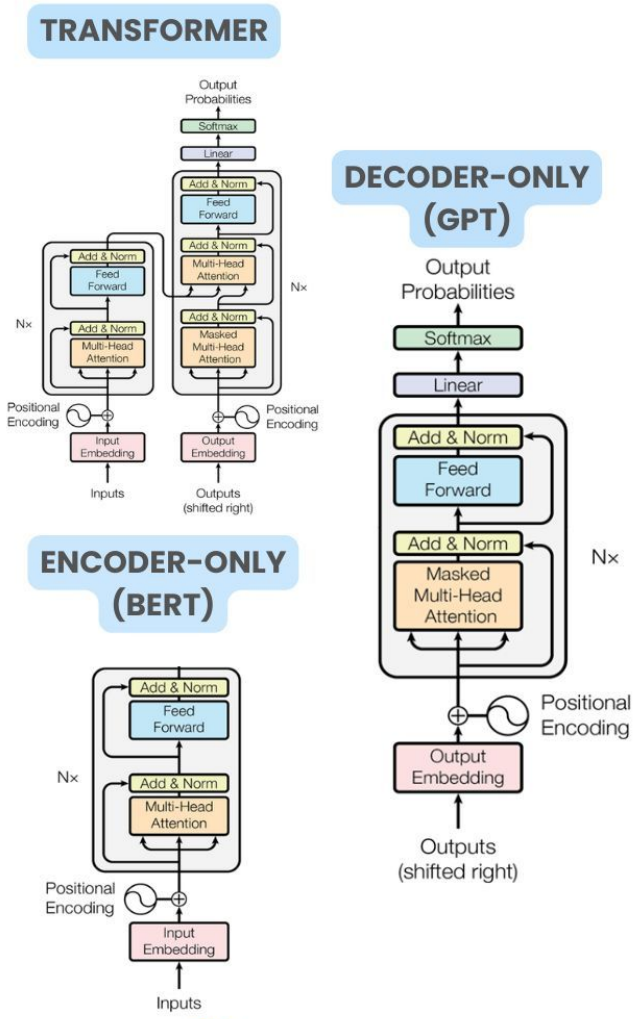
- Older models (RNNs) read one word at a time, so links between distant words had to survive many steps — and often didn't.
- Attention lets every word read every other word directly, in parallel.

Key innovation

- Self-attention connects distant tokens directly.
- Each token can weigh other tokens in the context, so long-range relationships no longer have to pass through many recurrent steps.

Why it scaled

- The same attention-plus-MLP block can be stacked and trained in parallel — that one block becomes GPT, BERT, Llama and Qwen.



VASWANI ET AL. 2017

https://poloclub.github.io/transformer-explainer/

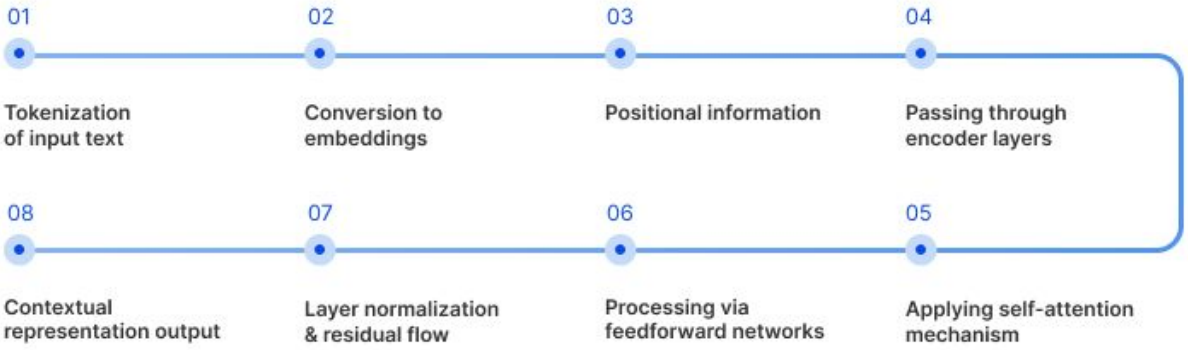
Inside the Transformer: one block, repeated

Encoder vs Decoder

- **Encoder (BERT-style):** reads both directions — classification, embeddings.
- **Decoder (GPT/Llama):** generates left-to-right, masked — chat and generation.
- Most modern LLMs are decoder-only.

Stack depth

- **GPT-2:** 12 layers · GPT-3: 96 layers · Llama 3 70B: 80 layers.
- Same block stacked deeper = more capacity.
- **Each block:** multi-head attention → layer norm → MLP → layer norm.



EMBL

ACT II

Inside a modern LLM

Now we open the hood: tokens, embeddings, attention, MLPs and the engineering tricks that make generation feasible.

Text becomes integer IDs and vectors.

Attention routes information; MLPs transform it.

Logits become the next token.

Most modern LLMs reuse the same decoder skeleton

What is shared

- Text becomes tokens, tokens become vectors, repeated transformer blocks update those vectors.
- a final projection scores the next token.

What changes between models

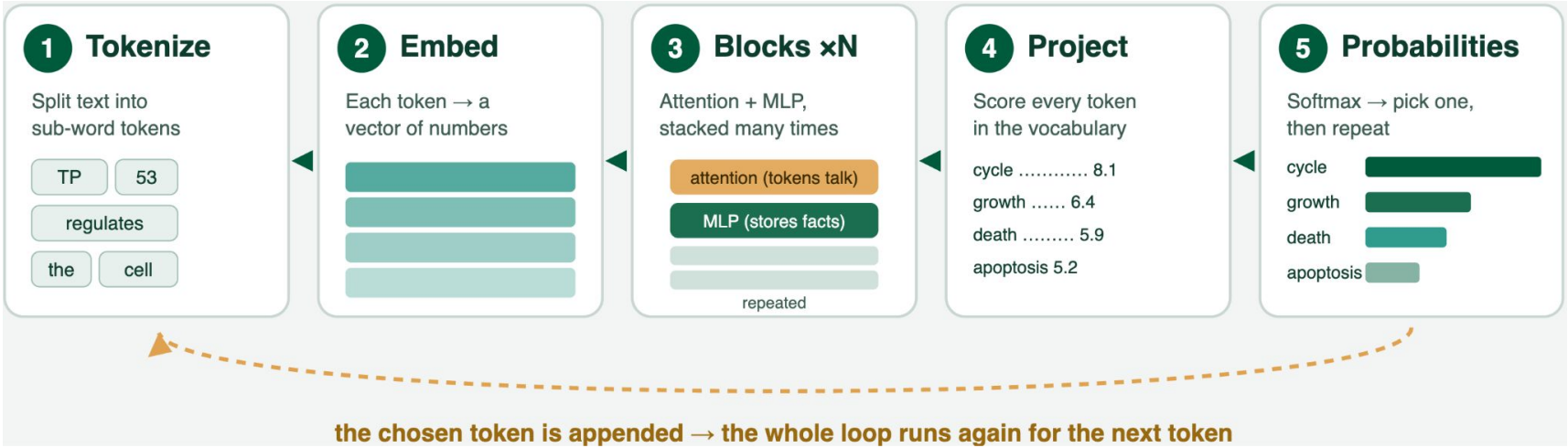
- Number of parameters, layers and context length.
- Training data mixture and filtering pipeline.
- Attention variants, KV cache design, MoE experts, quantization.
- **Post-training:** instruction following, preferences, tool use and reasoning.



https://bbycroft.net/llm

Inside the forward pass: five steps

Running example: “TP53 regulates the cell” → predict the next token



The model never “looks up” an answer.
It turns your text into numbers, reshapes them through stacked blocks, and outputs a probability over the next token — one token at a time

Where knowledge lives? How generation works

Attention — tokens talk to each other

- Moves information between positions so each word is read in context.
- Different heads learn different relationships (e.g. linking a pronoun to its noun).

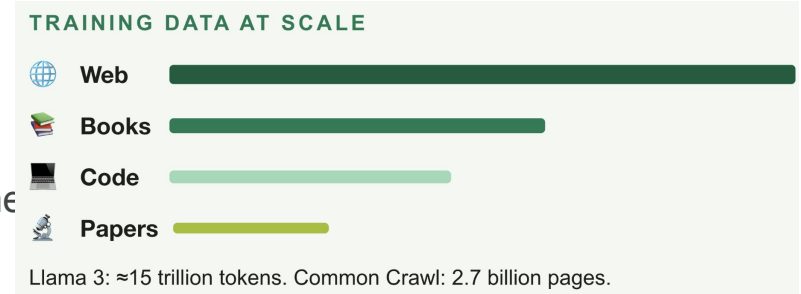
MLP — where facts are stored

- Transforms each position on its own; acts like a learned lookup table.
- Most of the model's parameters — and most of its stored world knowledge — live here.

Pretraining is next-token prediction at scale

The task

- **Labels are the text shifted by one** — predict the next token
- No human annotation needed; scales to the whole web for free
- **Cross-entropy loss:** how surprised was the model by the true next token?



Not an assistant yet

- A base model continues any plausible text. Post-training teaches it to follow instructions — that comes next.

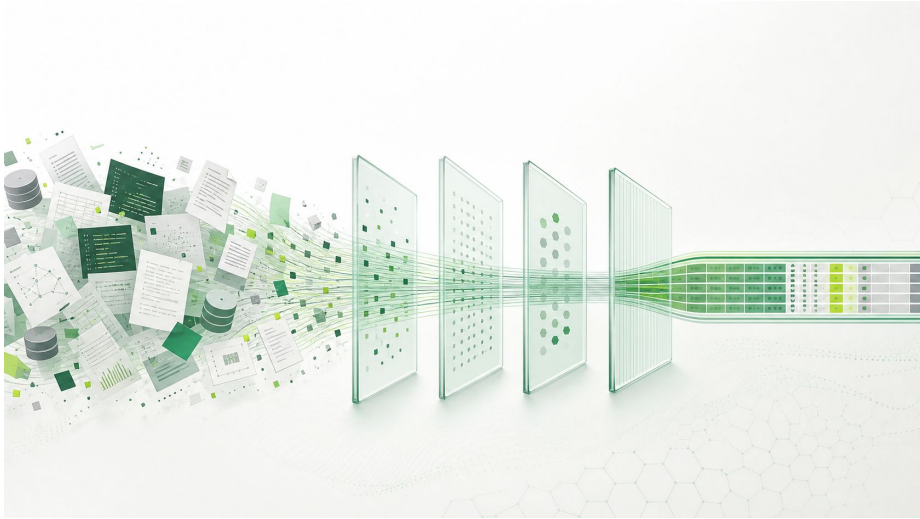
Where data comes from? Why filtering matters

Raw web is not training data:

- Common Crawl indexes 2.7 billion pages (2024)
- FineWeb extracts ≈44 TB of clean text after URL filtering, language ID, dedup and PII removal.

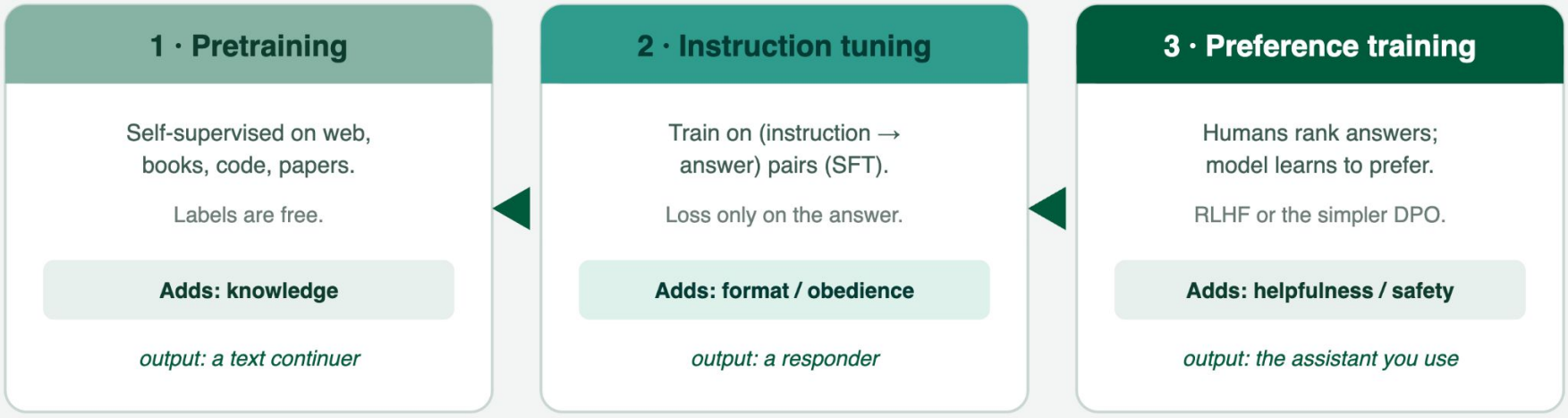
The funnel

- ≈240 trillion raw tokens → 1–6 trillion usable after quality filters.
- The model did not read the internet; it read the output of a large, opinionated data-cleaning pipeline.
- Modern model quality is as much data engineering as architecture.



Three stages turn a base model into an assistant

Same machine, three kinds of “studying” — each adds something different.



Think of it like studying:
Pretraining = reading the textbook exposition.
Instruction tuning = working through solved examples.
Preference training = doing practice problems and getting graded. Same student, three kinds of study.

Instruction tuning + few-shot prompting

Instruction tuning (SFT)

- Train on instruction → answer pairs.
- Apply loss mainly to answer tokens, not the prompt.
- Chat templates encode system / user / assistant roles.
- Result: a responder, not just a text continuer.

Prompting without retraining

- Few-shot prompting gives 2–3 examples in the context window.
- No weights change, but behavior can shift strongly.

Good examples often beat a longer instruction — a first taste of context engineering.

Post-training: teaching the model what behavior we want

Pretraining gives knowledge; post-training shapes behavior.

Method	Idea	What it adds
SFT	Show examples of good answers	Instruction following
Preference training	Prefer answer A over answer B	Helpfulness / safety
Verifier training	Reward answers that pass checks	Better math, code, tool use

Post-training does not create a new architecture. It changes how the same model behaves.

TEST-TIME COMPUTE

Reasoning models spend more compute while answering

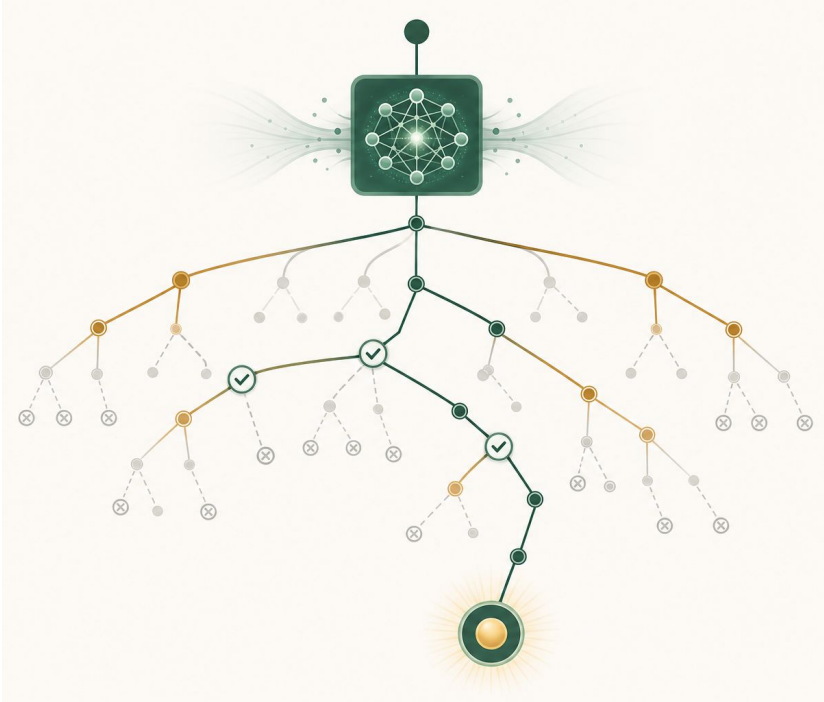
What changes

- No emitting the first plausible continuation
- Allocates extra tokens to explore, verify and revise intermediate steps.

Where it helps

- Math or logic where answers can be checked.
- Code where tests can run.
- Tool use where observations can confirm or reject a plan.

The tradeoff is latency and cost: more thinking tokens mean a slower, more expensive answer.



ACT V

Capability is not reliability

Before using LLMs for science, we need to understand what benchmarks measure and why fluent text can still be unsupported.

Benchmarks are useful instruments, not reality.

Public tests can leak into training data.

Fluent text is not the same as evidence.

Benchmarks are useful — and always proxies

What they do well

They make models comparable under fixed rules.

What they miss

A high score on a narrow task is not the same as reliable performance in your workflow.

Failure modes

- Distribution shift
- Metric gaming
- Benchmark contamination

Practical rule

Use public benchmarks as one signal; build fresh private evals for your real task.

Benchmark contamination: when the test leaks into training

Problem: Public benchmarks are often on the web.

Consequence: A model may look like it is reasoning, but partly be recalling test-like examples.

Mitigation

- Fresh private test sets
- Adversarial variants
- Decontamination checks
- Task-level evidence

Bottom line

- Do not trust leaderboards alone.

Hallucination is structural

Why it happens

- LLMs store compressed statistical memory, not a database.
- Training rewards plausible continuation, not verification.
- Risk is highest for rare, recent, or species-specific facts.

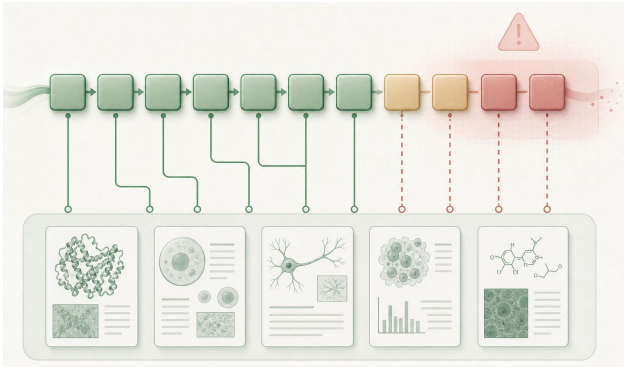
Biology example

An invented PMID, wrong gene symbol, or unsupported drug interaction.

What helps

- Retrieve evidence from papers or databases before answering.
- Require citations from a closed evidence set.
- Use tools for exact IDs, coordinates, arithmetic and code.

Hallucination is reduced, not abolished.



What does this mean for your work?

Because fluent ≠ true, scientific use needs grounding. That motivates everything next.

The problem: The model is equally confident whether or not it is right — it cannot tell evidence from a fluent guess.

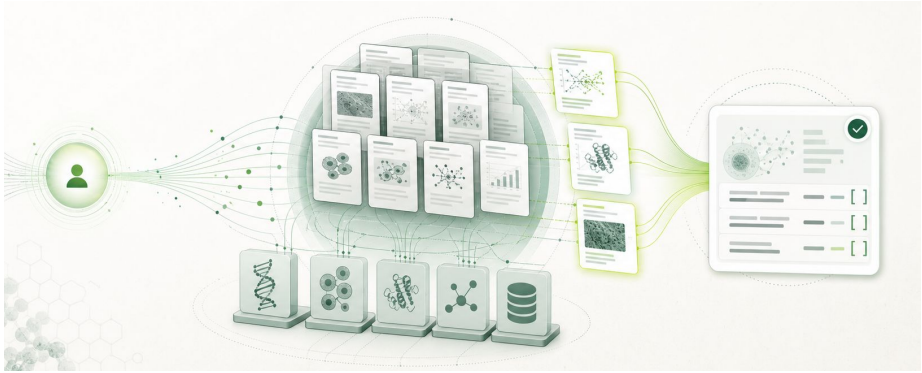


For a load-bearing claim — a gene function, a drug interaction — never accept weight-recall alone.

Parametric memory vs external memory

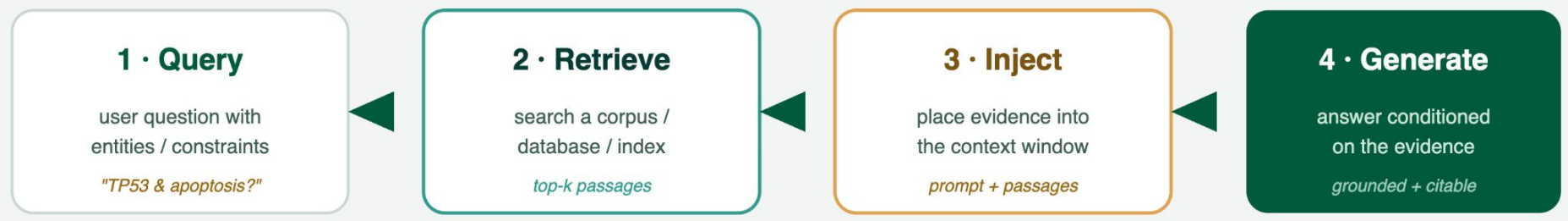
Weights are vague long-term memory; the context window is working memory you control.

- **Inside the weights (vague memory):** fast and fluent, but opaque, hard to audit, and frozen at training time.
- **Outside the model (working memory):** documents, databases and tools fetched at answer time — cited, updatable, rerunnable.
- **For load-bearing claims:** prefer evidence you **can inspect over fast weight-recall.**
- **The design choice:** when to trust the model's parametric memory vs when to retrieve.



RAG means: look up evidence before answering

Retrieve real documents, put them in the prompt, then answer from them.



RAG helps when facts matter
The model answers from retrieved papers, records, or database entries.

But RAG is not magic
Bad retrieval → bad answer.
Ignored context → fluent hallucination.

Scientific rule
Every load-bearing claim should point to a record you can inspect.

Three Ways to Answer a Question

“Which TP53 papers were published after 2023?”

1 · LLM only

Remembering

weights
(parametric memory)

Source: what it memorised in training

⚠ **Risk**

Outdated or hallucinated — may invent a plausible PMID.

2 · Retrieval (RAG)

Reading

weights + **retrieved** documents

Source: weights + evidence pulled into the prompt

✓ **Advantage**

Grounded in inspectable evidence — but can't act.

3 · Agent

Acting

weights | **retrieval** | **actions**

- query Europe PMC
- filter by date > 2023
- inspect the results
- generate citations

✓ **Can verify & cite**

Returns a checkable answer.

Capability grows: remembering → reading → acting

What is an agent?

The definition

- An agent is an LLM placed inside a harness: observe state → decide action → call tool → read result → repeat.
- **The LLM is the policy** — it decides what to do next.
- **Tools are the effectors** — file system, code runner, database, search, API.
- The harness manages the loop, memory and permissions.

What an agent can do that a chatbot cannot

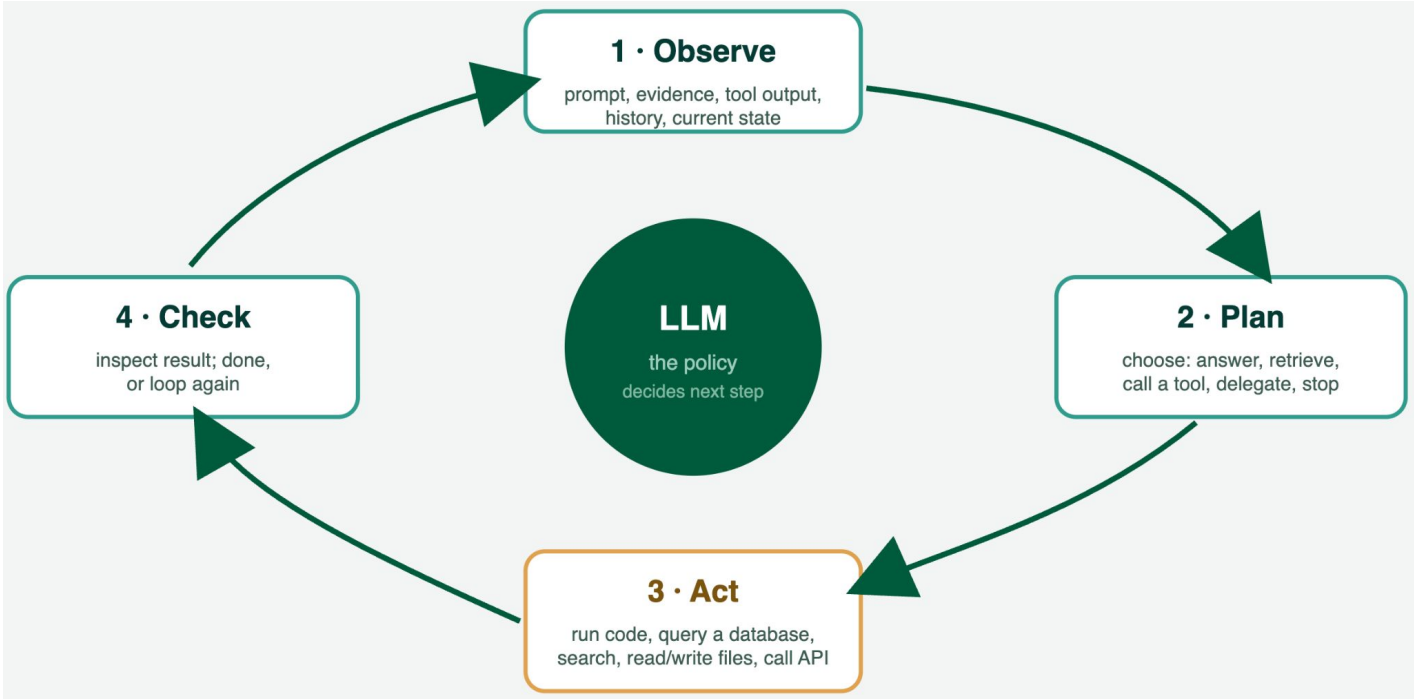
- Run multiple steps over time without human prompting.
- Call real tools and act on the results.
- Accumulate state across a long task.
- Delegate sub-tasks to other agents (multi-agent systems).

The agent loop: observe, plan, act, check

An agent is an LLM placed inside a harness that repeats this cycle.

Tools:

- Code runner
- File system
- Literature search
- Database / API
- Europe PMC, UniProt, PDB



harness = loop +memory + permissions

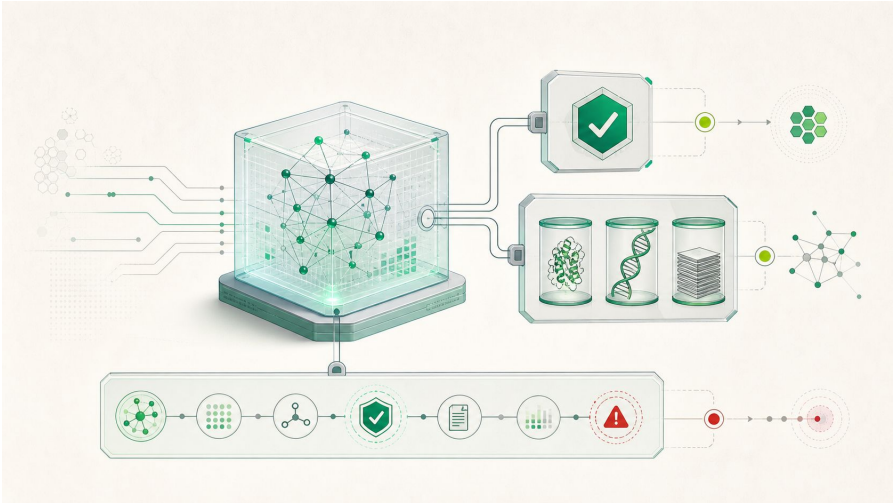
Agent safety is about the harness

Why agents are brittle

- Errors compound across steps.
- The model can act on wrong assumptions.
- External content can inject misleading instructions.

How to make them safer

- Restrict permissions (least privilege).
- Add human approval for critical actions.
- Sandbox and log all tool use.



LIVE DEMO · Scientific tool call

- **Question:** "Which proteins interact with TP53?"
- **Structured call:** get_protein_interactions("TP53")
- The harness executes a Python function and returns a structured observation.
- **Notebook** https://colab.research.google.com/drive/1KkSy7Ofjn_nST_7695D7Ri7NmLVCKuAy#scrollTo=liFgGmfR6ia8
- **Concept:** the model proposes an action instead of guessing; the tool result is deterministic and inspectable.

Scientific agents need scientific tools

Main claim

A useful scientific agent must query live, structured sources and return evidence.

Examples

- Literature: PubMed / Europe PMC
- Proteins: UniProt, STRING, PDB
- Chemistry: ChEMBL
- Code: analysis packages and pipelines

Scientific requirement

Answers need provenance, versioned data, reproducible queries, and citations.

Example

ToolUniverse exposes 1,000+ scientific tools.

PRACTICAL TAKEAWAYS

How to use LLMs in scientific work

EMBL

Prompting is Context Engineering

Weak prompt: "Tell me about TP53."

Strong prompt: "Summarize TP53's role in apoptosis using only the papers provided below.

For each claim:

- cite supporting evidence
- note uncertainty
- separate established findings from hypotheses

What improves performance?

- Clear task definition
- Explicit constraints
- Grounding material
- Desired output format
- Examples when available

The model cannot read your mind. Prompting is specifying the task environment.

PRACTICAL TAKEAWAYS

How to use LLMs well in scientific work

1. **Draft**
   a. Use for speed: code explanations, outlines, tests, abstracts.

2. **Ground**
   a. Use tools for facts: papers, databases, arithmetic, coordinates.

3. **Verify**
   a. Check rare, recent, species-specific, clinical, or unpublished claims.

4. **Reset**
   a. New topic = new chat. Stale context degrades quality.

5. **Protect**
   a. Do not put sensitive or unpublished data into public APIs.

Five take-home messages

1. **Next-token prediction is the anchor** — but scale, data, post-training and tools make it far more than autocomplete.

2. **A transformer is one block stacked and scaled;** models differ in weights, config and post-training, not machinery.

3. Data quality, scale, post-training and reasoning made it useful and steerable.

4. **Fluent ≠ reliable.** Hallucination is structural; evaluation needs fresh tasks, private data, not just leaderboards.

5. **Scientific value comes from grounded, auditable workflows** — models connected to papers, databases, code and tools.

A modern LLM is

- *a next-token predictor that scale made knowledgeable,*
- *post-training made helpful,*
- *reasoning made deliberate*
- *tools made able to act in the world.*

Luigi Sigillo · Petroni Lab · EMBL Rome

Thank you — questions welcome.

https://colab.research.google.com/drive/1S1t9zyUKtnQ7UOuUyuTeUS1SF7MsPZI_

Cheat-sheet

Token	a sub-word chunk; the unit the model predicts
Parameter	a learned number; models have billions
Embedding	a word/token as coordinates in meaning-space
Attention	tokens look at each other to share context
MLP	the layer where most facts are stored
Logits / softmax	raw scores turned into probabilities
Pretraining	next-token prediction over the whole web
SFT	fine-tune on (instruction → answer) pairs
RLHF / DPO	shape behaviour from human preferences
Context window	the model's working memory (what it can see)

KV cache	stored work so generation isn't recomputed
Hallucination	fluent text not backed by evidence
RAG	retrieve documents, then answer from them
Agent	an LLM in a loop that can use tools
Tool	a function the model can call (search, DB, code)
Skill	reference info pulled in only when needed
MCP	a shared standard for exposing tools to agents
LoRA	a small add-on that cheaply specialises a model
Reasoning model	spends extra tokens "thinking" before answering
Open vs closed	weights you can run locally vs API-only models